CHALLENGE 2026 · SPRINT 1 · MITOS E VERDADES NUCLEARES

Projeto Núcleo: Documentação do Banco de Dados

Modelo conceitual, lógico e físico (3FN), DER na notação de Barker e scripts Oracle

1. Contexto da solução

O Núcleo é um aplicativo mobile criado para combater a desinformação sobre energia nuclear por meio de jogos curtos. O aplicativo oferece o jogo Mito ou Verdade, no qual o jogador avalia afirmações sobre o tema, e o jogo Equilíbrio da cidade, no qual o jogador monta uma matriz energética e observa o resultado. Há ainda cadastro e login de usuários, um ranking de pontuação e uma área para manter o banco de perguntas.

O banco de dados foi modelado a partir dessas funcionalidades e registra apenas o que elas exigem: os usuários e a sua pontuação, as perguntas, as respostas dadas e as simulações da cidade. Conforme o plano do Sprint 1, a API Java trabalha com a entidade Pergunta e o console .NET trabalha com a entidade Usuario.

2. Funcionamento do banco de dados

O banco registra o que acontece no aplicativo. As entidades USUARIO e PERGUNTA são os cadastros de base: a primeira guarda os jogadores e a pontuação de cada um, e a segunda guarda o conteúdo do jogo Mito ou Verdade, mantido pela API Java. Esses registros mudam pouco, apenas quando um usuário se cadastra ou quando uma pergunta é criada, alterada ou removida.

As entidades RESPOSTA_USUARIO e SIMULACAO_CIDADE formam o histórico e crescem a cada jogada. Cada resposta no jogo Mito ou Verdade e cada simulação no jogo Equilíbrio da cidade gera um novo registro, vinculado ao usuário que jogou. Quando o jogador acerta uma pergunta ou conclui uma simulação, os pontos são somados ao atributo pontuacao de USUARIO, e o ranking corresponde à lista de usuários ordenada por esse atributo. Dessa forma, o histórico recebe apenas inserções, e a única informação alterada ao longo do uso é a pontuação.

3. Modelo conceitual

3.1 Entidades e atributos

O modelo conceitual possui quatro entidades. Nas tabelas a seguir, PK indica a chave primária e FK indica a chave estrangeira.

USUARIO: representa a pessoa que joga e aparece no ranking. É a entidade trabalhada pelo console .NET.

Atributo	Tipo	Obrigatório	Descrição
id_usuario (PK)	Número	Sim	Identificador único do usuário
nome	Texto	Sim	Nome exibido no ranking
email	Texto	Sim	E-mail de acesso; não pode se repetir
senha_hash	Texto	Sim	Senha armazenada de forma criptografada
pontuacao	Número	Sim	Total de pontos acumulados; inicia em zero
dt_cadastro	Data	Sim	Data de criação da conta

PERGUNTA: representa uma afirmação do jogo Mito ou Verdade. É a entidade trabalhada pela API Java.

Atributo	Tipo	Obrigatório	Descrição
id_pergunta (PK)	Número	Sim	Identificador único da pergunta
enunciado	Texto	Sim	Afirmação analisada pelo jogador
resposta	Texto	Sim	Resposta correta: V (verdadeiro) ou F (falso)
explicacao	Texto	Sim	Explicação exibida após a resposta
referencia	Texto	Não	Fonte que comprova a informação

RESPOSTA_USUARIO: representa o histórico de respostas, ou seja, o que cada usuário respondeu em cada pergunta e se acertou.

Atributo	Tipo	Obrigatório	Descrição
id_resposta (PK)	Número	Sim	Identificador único da resposta
id_usuario (FK)	Número	Sim	Usuário que respondeu
id_pergunta (FK)	Número	Sim	Pergunta respondida
resposta_dada	Texto	Sim	Escolha do jogador: V ou F
acertou	Texto	Sim	S se a resposta confere com a da pergunta, N caso contrário
dt_resposta	Data e hora	Sim	Momento em que a resposta foi dada

SIMULACAO_CIDADE: representa cada simulação feita no jogo Equilíbrio da cidade.

Atributo	Tipo	Obrigatório	Descrição
id_simulacao (PK)	Número	Sim	Identificador único da simulação
id_usuario (FK)	Número	Sim	Usuário que jogou
qtd_solar, qtd_eolica, qtd_nuclear, qtd_carvao	Número	Sim	Unidades escolhidas de cada fonte de energia, de 0 a 3
pontos_ganhos	Número	Sim	Pontos obtidos na simulação
dt_simulacao	Data e hora	Sim	Momento em que a simulação foi feita

3.2 Relacionamentos

O modelo possui três relacionamentos, todos do tipo um para muitos (1:N). Um USUARIO pode registrar várias RESPOSTA_USUARIO, e cada resposta pertence a um único usuário. Uma PERGUNTA pode ser respondida várias vezes, e cada resposta refere-se a uma única pergunta. Um USUARIO pode realizar várias SIMULACAO_CIDADE, e cada simulação pertence a um único usuário.

A entidade RESPOSTA_USUARIO é associativa: ela resolve o relacionamento muitos para muitos entre USUARIO e PERGUNTA, já que um usuário responde várias perguntas e uma pergunta é respondida por vários usuários. O caminho usuário, resposta e pergunta permite saber quais perguntas cada usuário respondeu e se acertou.

4. Justificativa das decisões de modelagem

Cada entidade foi escolhida porque uma funcionalidade do aplicativo depende dela. USUARIO sustenta o cadastro, o login e o ranking; PERGUNTA sustenta o jogo Mito ou Verdade e o cadastro de perguntas; RESPOSTA_USUARIO sustenta o histórico de acertos de cada jogador; e SIMULACAO_CIDADE sustenta o jogo Equilíbrio da cidade. Nenhuma tela do protótipo ficou sem dados de apoio, e nenhum dado foi incluído sem uma tela que o utilize.

A pontuação foi mantida como atributo de USUARIO porque o console .NET trabalha diretamente com esse campo e porque o ranking passa a depender de uma única consulta ordenada, sem somar o histórico a cada acesso. A entidade RESPOSTA_USUARIO foi criada à parte para evitar colunas repetidas em USUARIO e PERGUNTA e para registrar o que cada jogador respondeu e se acertou.

Os indicadores exibidos no jogo Equilíbrio da cidade, como emissão de carbono, custo e estabilidade, não são armazenados, pois são calculados a partir das quantidades de cada fonte de energia. Armazená-los criaria colunas que dependem de outras colunas não chave, o que contraria a terceira forma normal.

Ficaram fora do modelo, por não serem necessários ao Sprint Flash, os perfis de usuário, os níveis, o convite de amigos e a recuperação de senha.

5. Uso das entidades pelas APIs Java e .NET

A API Java e o console .NET utilizam uma entidade principal cada, e as demais entidades completam o modelo no banco de dados.

Entidade	Utilizada por	Finalidade
PERGUNTA	API Java	Cadastro, edição e exclusão, busca por texto do enunciado e pergunta aleatória
USUARIO	Console .NET	Cadastro, login com e-mail único, consulta e pontuação
RESPOSTA_USUARIO	API Java e console .NET	Registro das respostas e dos acertos que alimentam a pontuação
SIMULACAO_CIDADE	Console .NET	Registro das simulações do jogo Equilíbrio da cidade

O plano do Sprint 1 exige apenas a entidade PERGUNTA na API Java e a entidade USUARIO no console .NET. As entidades RESPOSTA_USUARIO e SIMULACAO_CIDADE completam o modelo de dados e poderão ser utilizadas pelas aplicações conforme a evolução da solução.

6. Diagrama Entidade-Relacionamento (notação de Barker)

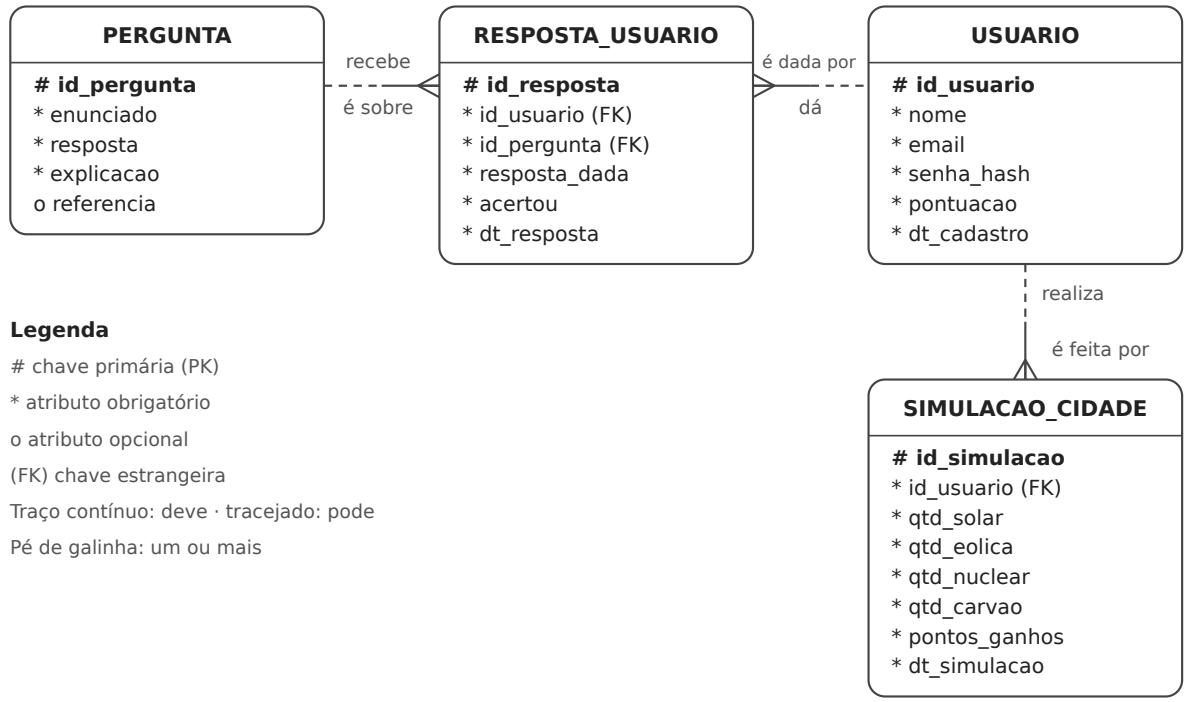


Figura 1 – DER do banco de dados do Núcleo na notação de Barker

No diagrama, as entidades são representadas por retângulos de cantos arredondados. O símbolo # marca a chave primária, o asterisco marca o atributo obrigatório e a letra o marca o atributo opcional. Nos relacionamentos, o traço contínuo indica participação obrigatória, o tracejado indica participação opcional e o pé de galinha indica o lado muitos.

Cada USUARIO pode dar uma ou mais RESPOSTA_USUARIO, e cada RESPOSTA_USUARIO deve ser dada por um e somente um USUARIO. Cada PERGUNTA pode receber uma ou mais RESPOSTA_USUARIO, e cada RESPOSTA_USUARIO deve ser sobre uma e somente uma PERGUNTA. Cada USUARIO pode realizar uma ou mais SIMULACAO_CIDADE, e cada SIMULACAO_CIDADE deve ser feita por um e somente um USUARIO.

7. Modelo lógico

O modelo lógico traduz as quatro entidades em tabelas relacionais, com suas colunas, chaves primárias (PK) e chaves estrangeiras (FK), sem ainda definir tipos de dados de um banco específico.

```
USUARIO (id_usuario PK, nome, email, senha_hash, pontuacao, dt_cadastro)

PERGUNTA (id_pergunta PK, enunciado, resposta, explicacao, referencia)

RESPOSTA_USUARIO (id_resposta PK, id_usuario FK, id_pergunta FK,
                  resposta_dada, acertou, dt_resposta)

SIMULACAO_CIDADE (id_simulacao PK, id_usuario FK, qtd_solar, qtd_eolica,
                  qtd_nuclear, qtd_carvao, pontos_ganhos, dt_simulacao)
```

Cada relacionamento 1:N do modelo conceitual tornou-se uma chave estrangeira no lado muitos. O atributo id_usuario de RESPOSTA_USUARIO referencia USUARIO, o atributo

id_pergunta de RESPOSTA_USUARIO referencia PERGUNTA, e o atributo id_usuario de SIMULACAO_CIDADE referencia USUARIO.

8. Modelo físico (Oracle)

O modelo físico define o tipo e as restrições de cada coluna para o banco Oracle. Os valores V/F e S/N são armazenados em CHAR(1) com restrição CHECK, e as datas utilizam os tipos DATE ou TIMESTAMP.

Tabela	Coluna	Tipo	Restrições
USUARIO	id_usuario	NUMBER(10)	PK
	nome	VARCHAR2(100)	NOT NULL
	email	VARCHAR2(150)	NOT NULL, UNIQUE
	senha_hash	VARCHAR2(255)	NOT NULL
	pontuacao	NUMBER(10)	NOT NULL, DEFAULT 0, CHECK (pontuacao >= 0)
	dt_cadastro	DATE	NOT NULL, DEFAULT SYSDATE
PERGUNTA	id_pergunta	NUMBER(10)	PK
	enunciado	VARCHAR2(500)	NOT NULL
	resposta	CHAR(1)	NOT NULL, CHECK (resposta IN ('V','F'))
	explicacao	VARCHAR2(1000)	NOT NULL
	referencia	VARCHAR2(300)	Opcional
RESPOSTA_USUARIO	id_resposta	NUMBER(10)	PK
	id_usuario	NUMBER(10)	NOT NULL, FK para USUARIO
	id_pergunta	NUMBER(10)	NOT NULL, FK para PERGUNTA
	resposta_dada	CHAR(1)	NOT NULL, CHECK (resposta_dada IN ('V','F'))
	acertou	CHAR(1)	NOT NULL, CHECK (acertou IN ('S','N'))
	dt_resposta	TIMESTAMP	NOT NULL, DEFAULT SYSTIMESTAMP
SIMULACAO_CIDADE	id_simulacao	NUMBER(10)	PK
	id_usuario	NUMBER(10)	NOT NULL, FK para USUARIO
	qtd_solar, qtd_eolica, qtd_nuclear, qtd_carvao	NUMBER(1)	NOT NULL, CHECK (valor BETWEEN 0 AND 3) em cada coluna
	pontos_ganhos	NUMBER(5)	NOT NULL, DEFAULT 0, CHECK (pontos_ganhos >= 0)
	dt_simulacao	TIMESTAMP	NOT NULL, DEFAULT SYSTIMESTAMP

Cada chave primária recebe uma sequence própria (seq_usuario, seq_pergunta, seq_resposta_usuario e seq_simulacao_cidade), utilizada na inserção dos registros. As chaves estrangeiras de RESPOSTA_USUARIO e SIMULACAO_CIDADE foram criadas com ON DELETE CASCADE, de modo que a exclusão de uma pergunta pelo CRUD da API Java, ou de um usuário, não seja bloqueada pelo histórico vinculado a eles.

9. Verificação da terceira forma normal (3FN)

As quatro tabelas atendem à 3FN, o que foi verificado em três etapas.

Forma normal	Regra	Verificação
1FN	Colunas atômicas e sem grupos repetidos	Cada coluna guarda um único valor. As quatro quantidades de energia são atributos distintos, um para cada fonte do jogo, e não uma lista repetida.
2FN	1FN e nenhuma dependência parcial da chave primária	Todas as tabelas têm chave primária de uma única coluna, portanto nenhuma coluna depende apenas de parte da chave.
3FN	2FN e nenhuma dependência entre colunas não chave	Cada coluna depende apenas da chave primária. Dados de outra tabela, como o nome do usuário ou o texto da pergunta, são obtidos pela chave estrangeira e nunca copiados.

Os indicadores do jogo Equilíbrio da cidade, como emissão de carbono, custo e estabilidade, não são armazenados, pois são calculados a partir das quantidades de cada fonte. Armazená-los criaria colunas dependentes de outras colunas não chave, o que violaria a 3FN.

Os atributos acertou e pontos_ganhos registram o que foi concedido em cada jogada e funcionam como histórico. Eles não são recalculados depois, e a soma dos pontos alimenta o atributo pontuacao de USUARIO.

10. Scripts DDL e DML

O arquivo Nucleo_Challenge_1Sem_2TDS_BD.sql reúne, em um único script, a criação e a carga do banco. A primeira parte (DDL) remove os objetos antigos, caso existam, e cria as quatro sequences, as quatro tabelas com suas chaves primárias, chaves estrangeiras e restrições NOT NULL, UNIQUE e CHECK, além de três índices nas chaves estrangeiras. A segunda parte (DML) insere cinco usuários, cinco perguntas, doze respostas e seis simulações da cidade, com dados coerentes com a solução: a pontuação de cada usuário corresponde a 100 pontos por resposta certa somados aos pontos obtidos nas simulações.

Duas consultas com JOIN respondem a perguntas da solução. A primeira lista as perguntas que cada usuário respondeu corretamente, unindo RESPOSTA_USUARIO, USUARIO e PERGUNTA.

```
SELECT u.nome AS usuario, p.enunciado AS pergunta, r.resposta_dada,
       TO_CHAR(r.dt_resposta, 'DD/MM/YYYY HH24:MI') AS respondida_em
  FROM resposta_usuario r
  JOIN usuario  u ON u.id_usuario  = r.id_usuario
  JOIN pergunta p ON p.id_pergunta = r.id_pergunta
 WHERE r.acertou = 'S'
 ORDER BY u.nome, r.dt_resposta;
```

A segunda monta o ranking de pontuação com a quantidade de simulações da cidade de cada usuário, unindo USUARIO e SIMULACAO_CIDADE.

```
SELECT u.nome AS usuario, u.pontuacao AS pontuacao_total,
       COUNT(s.id_simulacao) AS qtd_simulacoes,
       NVL(SUM(s.pontos_ganhos), 0) AS pontos_da_cidade
  FROM usuario u
  LEFT JOIN simulacao_cidade s ON s.id_usuario = u.id_usuario
 GROUP BY u.id_usuario, u.nome, u.pontuacao
 ORDER BY u.pontuacao DESC, u.nome;
```